Universidad de Valparaíso — Escuela de Ingeniería Informática
Taller de Desarrollo de Software Conducido por Agentes de IA

MIRA

Entrega 2: incremento funcional con desarrollo conducido por especificación (SDD)

Grupo	Número no asignado
Integrantes	Nikolas Lagos; Franko Moraga; Dante Chaves; Felipe Caru
Representante del cliente (Entrega 2)	Nikolas Lagos. *Ejerció el mismo rol en la Entrega 1; la excepción a la regla de rotación del enunciado (cap. 5) se registra como DEC-16 y se discute en 6.2.*
Método declarado	SDD (Spec-Driven Development)
Herramienta y versión	GitHub Spec Kit, versión más reciente disponible a octubre de 2026, instalada sin fijar versión (CLI Spec Kit 1.2.2.dev0, Python 3.13.15, Windows)
Agentes de IA usados	Copilot (Nikolas Lagos, Dante Chaves, Felipe Caru) y Antigravity (Franko Moraga)
Repositorio	`https://github.com/Frxnks/Entregas_DSLLMs` Clonar: `git clone https://github.com/Frxnks/Entregas_DSLLMs.git` (pasos de ejecución en el Anexo G)
Versión evaluada	etiqueta `v2.0` del repositorio
Fecha de entrega	lunes 19 de octubre de 2026, 14:00 (plazo prorrogado por el profesor)

Archivo de entrega: `MIRA-Lagos-Moraga-Chaves-Caru.pdf` (sin número de grupo, que no fue asignado)

Índice de secciones

6.1 Resumen ejecutivo **2**

6.2 Introducción y selección de funcionalidades **3**

6.3 Aplicación del método **4**

6.4 Implementación **5**

6.5 Pruebas y verificación **7**

6.6 Comparativa con la Entrega 1 **8**

6.7 Discusión **11**

6.8 Conclusiones **12**

Anexo A. Trazabilidad requisito → especificación → tarea → commit → prueba **13**

Anexo B. Bitácora de decisiones **14**

Anexo C. Evidencia del trabajo con el agente **15**

Anexo D. Capturas y evidencia **16**

Anexo E. Guía simple: qué es SDD y qué técnicas enseña el curso **18**

Anexo F. Dónde se usó cada técnica en MIRA **21**

Anexo G. Cómo obtener y ejecutar el repositorio **23**

Anexo H. Sesión de aceptación: criterios de la E1 y evidencia disponible **24**

6.1 Resumen ejecutivo

Qué se construyó. Una aplicación web en español para revisar los reembolsos de gastos médicos que el sistema no puede resolver solo. Un primer módulo calcula un puntaje de confianza y decide si el caso se aprueba solo o pasa a una persona; un segundo módulo muestra a esa persona los casos pendientes, el motivo, la evidencia y el tiempo restante, y guarda su decisión. MIRA nunca rechaza por sí sola: todo caso dudoso o con sospecha de fraude llega a un humano.

Método. Se partió del análisis de la Entrega 1 y se escribió y revisó una especificación antes de generar el código (GitHub Spec Kit).

Tres cambios respecto de la Entrega 1. (1) Se eliminó el puntaje de corte bajo el cual el sistema rechazaba solo: la Entrega 1 lo suponía y la regla de no rechazo automático lo prohíbe. Ahora todo caso desfavorable pasa a una persona y el fraude tiene prioridad. (2) El plazo de "3 días hábiles" se midió en horas hábiles reales (lunes a viernes, 08:30 a 18:30, sin feriados) y, al vencer, el caso pasa a un supervisor en vez de cerrarse. (3) Se agregó un bloqueo temporal de 15 minutos para que dos personas no resuelvan el mismo caso a la vez.

Estado y recomendación. Las dos funciones corren con casos ficticios y 172 pruebas automáticas pasan, pero tres exigencias de operación (disponibilidad mensual, retención y cifrado de la evidencia, procesamiento asíncrono) no están demostradas. Antes de un piloto: probar con liquidadores reales y una base de datos de producción, y validar con el cliente la sugerencia de aprobar que MIRA muestra cuando el puntaje es bajo y no hay fraude.

6.2 Introducción y selección de funcionalidades

Contexto. MIRA (DPRIME SpA) automatiza la evaluación de solicitudes de reembolso. La Entrega 1 produjo requisitos (18 RQF y 8 RQNF), una bitácora de decisiones (DEC-01 a DEC-12), hallazgos (H-01 a H-11) y un backlog de 13 historias. Esta entrega los usa como entrada de un método SDD y comprueba qué sobrevive al llegar a código.

Funcionalidades. F4 Puntaje y decisión por umbrales y **F1 Bandeja de revisión asistida**, combinación citada en el enunciado (4.2). *Conexión:* F4 produce el puntaje, la causal y el estado "derivado" que F1 consume para poblar la bandeja y mostrar la sugerencia (`ScoringModule` productor, `ReviewTrayModule` consumidor). *Riesgo:* F4 concentra las reglas más costosas si fallan (nunca rechazo automático, DEC-04; el fraude prima sobre el puntaje; solo la aprobación automática es facturable, DEC-01) y F1 es donde esas reglas se vuelven visibles y auditables para el operador. *Cruce con la E1:* el umbral de aprobación era el hallazgo H-11 (la E1 reconoce que el 85 era solo la cifra de la demo y el cliente pedía 90), resuelto en DEC-11 como parámetro configurable con valor inicial 90.

Método y stack. SDD con GitHub Spec Kit (versión en la portada). Constitución `.specify/memory/constitution.md` (v2.0.0) y especificaciones en `specs/001-bandeja-revision-asistida` y `specs/002-puntaje-decision-umbrales`. Stack: NestJS 10 (módulos separados productor/consumidor), Prisma 5.22 sobre PostgreSQL 16 (restricciones y transacciones para el registro inalterable), Next.js 14 (interfaz en español) y Docker Compose (arranque con un comando, requisito del README).

Equipo. Nikolas Lagos (Copilot), Franko Moraga (Antigravity), Dante Chaves (Copilot) y Felipe Caru (Copilot).

Representante del cliente. Nikolas Lagos, quien también lo fue en la E1. Esto contradice la regla de rotación del enunciado (cap. 5); se registró como DEC-16 con su riesgo (sesgo de aceptación) y su mitigación: la sesión de aceptación la firma el equipo con la lista de criterios de la E1 y no solo el representante.

6.3 Aplicación del método

Fase (commit)	Artefacto y entrada	Qué cambió el equipo sobre lo generado
Constitución (`03d39bf`)	`constitution.md` v2.0.0 con los DEC de la E1 aplicables	Se añadieron las decisiones al archivo (mensaje del commit)
Especificar (`184d9a3`)	`spec.md` de F1 y F4 (393 líneas) desde RQF/RQNF de la E1 y *Descripción del proyecto*	Sin corrección propia; el agente agregó 5 elementos sin fuente (ver A3 en 6.6)
Aclarar (`8310291`)	5 preguntas, 34 líneas añadidas y 15 eliminadas en las specs	Respuestas del representante; 2 de 5 propuestas del agente rechazadas
Plan (`a334b86`)	`plan.md`, `data-model.md`, `quickstart.md` (3.110 líneas en `specs/`)	Revisión por análisis cruzado (fila siguiente)
Tareas (`253fd9a`)	`tasks.md`: 58 tareas	Pasaron a 61 tras el análisis
Analizar (`0b151a5`)	11 hallazgos de consistencia	Corrección del plan en `cb630a3`
Implementar	7 commits de avance (`6275887` a `450e2ff`)	Corrección de arranque en `1f32bef`

DEC de la E1 traducidas a la constitución. (a) **DEC-04** ("no se emiten rechazos automáticos") pasó a ser el Principio Constitucional II y se amplió: al vencer el plazo el caso se escala y no se cierra, y ante causales concurrentes el fraude prima sin generar rechazo (Q1 y Q4). (b) **DEC-11** (umbral configurable, 90 puntos) se amplió con un piso de seguridad de 60 puntos en señales documentales (Q2, RQF-022).

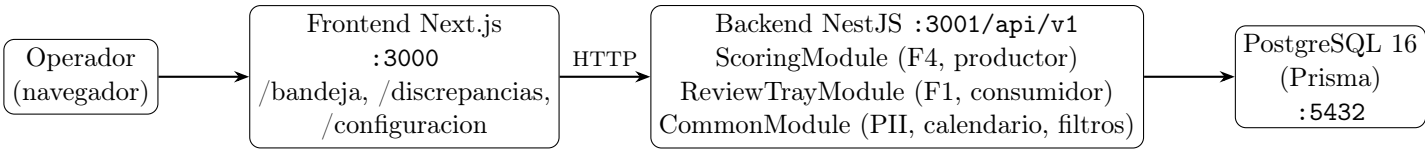
Preguntas del agente (clarify) y respuesta. Q1 acción ante plazo vencido → escalar a supervisión (DEC-04/DEC-13). Q2 mínimos por señal → piso de 60, editable. Q3 cómputo del plazo → horas hábiles de Chile. Q4 causales concurrentes → jerarquía fraude, mínimo de señal, puntaje. Q5 concurrencia → lectura libre más bloqueo de 15 minutos con confirmación del primero en enviar (DEC-14). Cada respuesta cita la fuente (*Descripción del proyecto*, ENT3/ENT4, LLMS) en `clarificacion_cambios.md`.

Artefacto rechazado o corregido, antes y después. *Antes:* el borrador de RQF-021 asumía un cronómetro continuo 24/7. *Por qué se rechazó:* un caso ingresado el viernes por la tarde vencería durante el fin de semana, contra lo dicho en ENT4. *Después:* plazo en horas hábiles (08:30–18:30, sin feriados) con escalamiento a supervisión (DEC-13). Mismo patrón con el bloqueo exclusivo de un solo usuario propuesto por el agente, reformulado por el representante como lectura concurrente más bloqueo de 15 minutos (DEC-14). Además, tras el análisis cruzado se corrigió el plan para incluir el servicio de enmascaramiento de datos personales, que el plan original no declaraba pese a DEC-06 (`cb630a3`; contenido generado por el agente a partir del hallazgo y aceptado por el equipo).

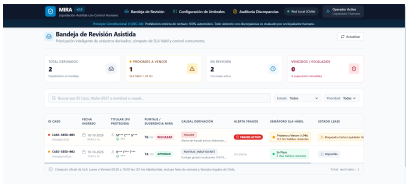
Valor y costo por fase. *Mayor valor:* aclarar, que corrigió dos propuestas del agente y creó DEC-13 y DEC-14, y analizar, que dio 11 hallazgos y 3 tareas más. *Mayor costo relativo:* el plan, con 3.110 líneas añadidas en `specs/` frente a 355 de las tareas, difíciles de revisar a fondo y con una sola corrección posterior (`cb630a3`). El material del curso estima un costo de 20 a 40 % más de tokens por funcionalidad frente a pedir por chat, a cambio de menos ciclos de corrección.

6.4 Implementación

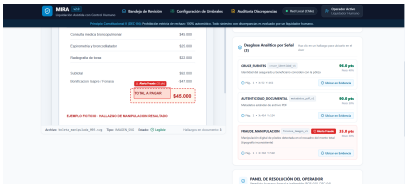
Arquitectura.



Modelo de datos efectivamente implementado (según `data-model.md`). Cambios frente al modelo de la E1: `Umbral` sin `valorRevision` y con `minimoBasalSeguridad`; `Caso` con `sugerenciaMIRA`, `marcaDiscrepancia`, `fechaVencimientoSLA`, `operadorBloqueoId` y `bloqueoExpiracion`; `Senal` ligada a `Evidencia` con `ubicacionHallazgo` (JSONB); `Evidencia` con `hashSha256`; tablas de solo inserción `Decision` y `AuditoriaUmbral`. *Interfaz* (más capturas en el Anexo D):



Bandeja de revisión



Detalle con evidencia y señales



Panel de resolución

Estado real (verificado el 18-oct-2026).

Funciona	Evaluación de puntaje por API con cuatro escenarios (aprobación automática, fraude, mínimo de señal, puntaje insuficiente), sin rechazo automático; bandeja con filtros y semáforo de plazo hábil; detalle con visor de evidencia y desglose de señales; bloqueo de caso por otro operador; registro de decisión con justificación (en la comprobación manual, `CASO-SEED-002` pasó a `RESUELTO`); cambio de umbral 90→88→90 con fila de auditoría de usuario y fecha. Comprobación manual hecha sin capturas.
Parcial	(i) Las pruebas de interfaz interceptan el bloqueo y la decisión, y las de backend sustituyen Prisma por mocks: no validan la persistencia en PostgreSQL. (ii) Playwright: 6/6 en un clon aislado con datos limpios; 2/6 sobre la base compartida ya modificada (faltaba `CASO-SEED-002`). (iii) `CASO-SEED-002` pondera 74,25 pero persiste 74,0. (iv) RUT: el *seed* guarda un valor ya enmascarado y el backend lo enmascara otra vez; corrección propuesta, no aplicada. (v) Sin cobertura demostrada: RQNF-002 (procesamiento asíncrono), RQNF-004 (cifrado y retención) y RQNF-006 (disponibilidad 99,9 %).
Fuera	F2, F3, F5 a F9; motor de análisis real (simulado, enunciado 4.3); SSO y conectores.

Autoría aproximada (líneas añadidas más eliminadas por commit, `autoria.md`):

Categoría	**Commits**	**Líneas**	**%**
Agente, sin cambios	36 commits generados por el agente (Spec Kit, avances de fase, evidencia `[IA]`)	42.802	99,47
Equipo corrigió al agente	`1f32bef`: arranque con Compose (Dockerfile, README, *seed*)	225	0,52
Equipo escribió	`46a31b1`: commit inicial	2	<0,01

Salvedades: el reparto es por commit y mide cambios de línea, no el tamaño final; `otros` (19.915 de 43.029 líneas) incluye el andamiaje de Spec Kit. La clasificación de `1f32bef` como corrección del equipo sobre código del agente es una lectura de este informe (confirmada por el equipo; `autoria.md` la registra como Equipo). El trabajo del equipo no fue escribir código sino dirigir al agente, responder como cliente, aceptar o rechazar y verificar.

6.5 Pruebas y verificación

Automatizadas. Backend: 16 suites y 172 pruebas aprobadas; subconjunto *backend e2e*: 3 suites y 41 pruebas (ya incluidas en las 172); verificación de tipos del frontend sin errores; Playwright: 6/6 en clon aislado y 2/6 en la base compartida (ver 6.4). `matriz_pruebas.md` enlaza 18 archivos de prueba con sus RQF/RQNF y sus PF/PX de la E1.

Derivación y reescrituras. Casos de la E1 no automatizados o no reproducidos fielmente: PF-06b (sin prueba ni criterio en la spec F4); PF-02 (se prueba una evidencia ilegible, no el expediente de tres archivos); PF-07 (no todas las pruebas usan los 45 puntos del caso original); PF-09/PF-20 (no se compara la posición visual del hallazgo); PF-10/PF-11 (la interfaz simula la respuesta); PF-21/PF-24 (sin notificación real al supervisor ni carrera real entre procesos); PX-07 y PX-08 de F4 (sin prueba de carga ni medición del desglose). PF-20 a PF-24 y PX-08 F1/F4 son criterios nuevos de las specs de la E2. *Advertencia de la pauta:* la matriz marca las pruebas más cercanas a la implementación que a la aceptación (servicios con Prisma simulado, prueba de bounding box sin geometría, escenarios de scoring con datos propios que pueden diferir del *seed*).

Excepción y E2E. Flujo de excepción: `fraud-precedence.spec.ts` (fraude con puntaje alto → derivado) y las pruebas de bloqueo y colisión (`concurrency.spec.ts`). E2E por interfaz: `review-flow.e2e.spec.ts` (Playwright) recorre bandeja, expediente, motivo corto y decisión; para F4, `quickstart-validation.spec.ts` y `scoring-quickstart.spec.ts` recorren solicitudes HTTP y la verificación manual de umbrales cubre la interfaz.

Verificación de F4 por API (`verificacion_F4.md`, `persistir=false`, 0 filas guardadas): A, dos señales 95 y 93 → 94 y aprobado automático; B, fraude con puntaje 99 → derivado por fraude; C, puntaje 96,8 con una señal bajo su mínimo → derivado por mínimo de señal; D, puntaje 80 sin fraude → derivado por puntaje insuficiente. En ninguno hubo rechazo automático. El desglose de `CASO-SEED-002` no cumplió la consistencia exacta (74,25 frente a 74).

Extra-funcional medible. PX-08 (RQNF-008): la carga de `/bandeja` midió 477 ms frente al umbral de 2 s que la E1 fija para cada pantalla (clon aislado; 476 ms en la base compartida antes de fallar la prueba). Mide el tiempo de carga y no la usabilidad sin capacitación, que se cubre aparte. Además, `extra-functional.e2e-spec.ts` mide latencia y llamadas concurrentes con servicios simulados, sin demostrar infraestructura real.

Clon limpio (`verificacion_clon_limpio.md` y repetición del equipo el 19-oct-2026): siguiendo solo el README con un volumen nuevo, los tres servicios quedaron en ejecución (la base *healthy*), la migración `0_init` se aplicó, el *seed* cargó los tres casos y la bandeja los muestra. La construcción tomó 154 s y el arranque unos 14 s: cerca de 3 minutos en total, bajo el límite de 15 minutos. Las capas de dependencias estaban en caché; no se probó en un equipo distinto y sin caché. Un primer intento del agente en un clon aislado falló porque el *seed* corrió antes de terminar las migraciones; se corrigió el README para esperar el mensaje de migración (`a89ae03`).

Usabilidad. No se realizó una sesión observada con una persona externa sin experiencia previa (formulario `prueba_usabilidad.txt` sin completar). Por eso PX-08 se registra como *Rechazado por falta de evidencia*: no demuestra un fallo de usabilidad, solo que no hay prueba. Lo único medido es la carga de `/bandeja` en 477 ms. El protocolo para repetirla quedó en la hoja de aceptación.

Sesión de aceptación. Registro de 11 criterios funcionales (PF) y 8 no funcionales (PX) de la E1 en el Anexo H, contrastados con la evidencia automatizada. Resultado de lo marcado en la hoja: 5 de 19 criterios. Rechazados: PF-06b y PX-08, ambos por falta de evidencia. Aceptados con observación: PX-02, PX-04 y PX-06, cuyas observaciones dicen que no se demostró el escenario completo. Los otros 14 criterios quedaron sin marcar y no se declaran aceptados. La hoja no registra fecha ni asistentes.

6.6 Comparativa con la Entrega 1

Convención: los DEC se citan con la numeración del Anexo (bitácora) de la E1, DEC-01 a DEC-12; las fichas del cuerpo de la E1 usan una numeración desplazada que ese informe reconoce en su Discusión. DEC-13 a DEC-16 son de esta entrega. El enunciado exige "dos tablas" y "tres indicadores" sin definirlos: se adoptan las definiciones siguientes como supuesto del equipo.

Tabla 1 (6.6.1). Clasificación de cambios en requisitos (universo: 18 requisitos de la E1 en F1/F4, 10 RQF y 8 RQNF).

Clase	N	Requisitos y motivo	Origen
Sin cambio	14	RQF-002, 004, 005, 009, 010, 011 y RQNF-001 a 008 conservan su intención	–
Modificado	4	RQF-006 se amplía a mínimos por señal; RQF-007 y RQF-008 prohíben además el cierre desestimatorio y fijan causales; RQF-015 pasa de Should a Must	Cliente (Q1, Q4); equipo
Nuevo	5	RQF-020 ubicación del hallazgo (ENT5); RQF-021 plazo hábil y escalamiento (Q1, Q3); RQF-022 mínimo por señal (Q2); RQF-023 precedencia de fraude (Q4); RQF-024 concurrencia (Q5)	Agente propone; cliente decide
Descartado	1	RQF-018 (migración de pilotos), ya Won't en la E1	E1
Diferido	7	RQF-001, 003, 012, 013, 014, 016, 017, fuera de F1/F4	Equipo (alcance)

Tabla 2. Supuestos de la E1 invalidados y quién los detectó.

N	Supuesto y reemplazo	Detectó
1	Umbral inferior y rechazo automático → se elimina `valorRevision` (DEC-04)	Equipo y cliente
2	Plazo continuo 24/7 → horas hábiles y escalamiento (DEC-13)	Cliente
3	Bandeja monousuario → lectura libre y bloqueo de 15 min (DEC-14)	Agente (alerta), cliente (solución)
4	Evidencia sin ubicación del hallazgo → relación señal–evidencia y coordenadas	Equipo
5	Sugerencia de MIRA no persistida → se guarda `sugerenciaMIRA`	Agente
6	F4 y F1 como un proceso síncrono → productor y consumidor	Equipo y agente
7	Inalterabilidad solo declarada → tablas de solo inserción	Equipo

Indicadores (definición propia).

Indicador	Fórmula y lectura	Valor
Cobertura de trazabilidad	Must de la E1 en alcance con cadena documental completa (requisito, spec, tarea, commit, prueba relacionada) / Must en alcance (9 RQF y 6 RQNF). Mide enlaces, no aceptación: RQNF-002, 004 y 006 y PF-06b no demuestran su criterio	15/15 = 100 %
Estabilidad de requisitos	Requisitos sin cambio / requisitos de la E1 en alcance (más 4 modificados, 5 nuevos, 1 descartado)	14/18 = 77,78 %
Proporción de autoría	Líneas del agente / líneas totales; complemento: equipo	99,47 % agente; 0,53 % equipo

A. Requisitos.

A1. Cambiaron 4 y se agregaron 5 (Tabla 1); los más relevantes surgieron de aclarar y no del enunciado: plazo hábil, bloqueo, mínimo por señal.

A2. Los 15 Must del alcance llegaron a especificación, tarea y una prueba relacionada. Quedan sin prueba que demuestre el criterio RQNF-002, RQNF-004 y RQNF-006 (requieren infraestructura real) y PF-06b (no figura en la spec F4).

A3. El agente añadió 5 elementos sin fuente: alerta a menos de 24 h con colores (aceptado, hace verificable RQF-021), nombres de estados (aceptado, para tipar el modelo), piso de 60 puntos (aceptado por el cliente), cronómetro 24/7 (rechazado, contradice ENT4) y bloqueo monousuario (rechazado, reformulado). Después se aceptó DEC-15: sugerir aprobar con puntaje bajo y sin fraude, siempre con decisión humana.

A4. No se automatizaron RQNF-002, 004 y 006 (y solo parcialmente 001, 003 y 005). Están formulados a nivel de operación (99,9 % mensual, retención de 5 años, residencia de datos), que una prueba dentro de un incremento no verifica: es una limitación del alcance y del método, que termina en implementar, más que un defecto de redacción; les faltaba un escenario medible a escala de incremento.

B. Supuestos y decisiones.

B1. Se invalidaron 7 supuestos (Tabla 2): 4 los detectó el equipo, 3 el cliente y 3 el agente (algunos en conjunto).

B2. De DEC-01 a DEC-12, 10 quedaron intactas y 2 se ampliaron (DEC-04, DEC-11); DEC-13 y DEC-14 son nuevas. El modelo de la E1 contradecía DEC-04 (umbral inferior). DEC-07 (ingesta asíncrona) figura intacta, pero no hay cola ni prueba asíncrona (RQNF-002): queda sin cumplir y sin declararse como desviación en la clarificación.

B3. De 5 ambigüedades que el agente resolvió solo, 3 coincidieron con el cliente (fraude primero, piso de 60, escalar al vencer) y 2 no (reloj 24/7 y bloqueo monousuario).

C. Inconsistencias.

C1. Abiertos desde la E1: H-03 (retención) y H-08 (sistema legado), pendientes de validar con el cliente. Nuevos al implementar: puntaje de `CASO-SEED-002` (74,25 frente a 74,0), doble enmascaramiento del RUT y la numeración desplazada de DEC en la propia E1.

C2. La arquitectura contradice DEC-07 y RQNF-002 en lo asíncrono (ver B2).

D. Modelo y calidad.

D1. Sí cambió: ver 6.4 (`sugerenciaMIRA`, bloqueo, ubicación del hallazgo, hash, tablas de solo inserción, sin `valorRevision`).

D2. No todos eran ejecutables tal cual: se reescribieron o no se pudieron automatizar PF-02, PF-06b, PF-07, PF-09/10/11 y PX-01 a PX-08 (6.5).

E. Proceso.

E1. Casi todo lo escribió el agente (99,47 % de líneas, Tabla de 6.4). El equipo contribuyó con respuestas del cliente, decisiones, y una corrección de arranque; se evidencia con `git log` y `autoria.md`.

E2. Tres cosas: verificar desde el inicio la numeración de las decisiones y las referencias internas, algo que la propia E1 ya advertía; escribir los RNF con un escenario medible a escala de incremento (por ejemplo, un tiempo de respuesta con un tamaño de carga) para poder automatizarlos; y separar en commits propios las correcciones humanas desde el primer día, para que el diff muestre qué aportó el equipo.

6.7 Discusión

Estructura útil y burocracia. La aclaración rindió: de cinco propuestas del agente, el cliente corrigió dos que habrían entrado al código (reloj 24/7 y bloqueo monousuario). El análisis cruzado encontró 11 hallazgos antes de programar. En cambio, el plan (3.110 líneas) es difícil de revisar a fondo y casi todo se aceptó con poca revisión visible: solo hay un commit de corrección sobre las specs.

Lo que el agente dio por bueno sin estarlo. La auditoría posterior encontró coberturas "100 %" sin respaldo, sumas de líneas que no cuadraban, una guía con un ejemplo que no coincidía con los datos y un plazo de "72 horas hábiles" cuando el código usa 30. Las pruebas pasaban, pero la aplicación no arrancaba desde cero (el contenedor no hallaba su archivo de arranque) y 14 de 16 suites fallaban sin generar el cliente de Prisma. "Pruebas en verde" y documentación generada no son evidencia de que el entregable funcione: lo es arrancar desde un clon limpio.

De la especificación al código. Se perdieron cosas entre ambos: PF-06b no llegó a ninguna prueba; RQNF-002, 004 y 006 quedaron sin demostrar; el *seed* no cumple la fórmula del propio puntaje y el RUT se enmascara dos veces. Ninguna estaba en la spec como riesgo.

Frente al prompting directo de la E1. En la E1 se usaron cinco técnicas de prompting (anclaje en fuentes, salida estructurada, planificar y ejecutar, verificación en cadena y juez con rúbrica) y el modelo igual numeró mal su propia bitácora, sin que la revisión superficial lo notara. En la E2 esas mismas técnicas siguieron presentes dentro de cada etapa y la especificación les dio un marco: la constitución fijó las reglas, cada etapa produjo un archivo versionado y revisable, y el análisis cruzado hizo de control. Se complementan: las técnicas mejoran cada respuesta; SDD ordena el recorrido completo. El resultado es más ordenado y trazable, pero *no más rápido*: hubo más artefactos que revisar (3.110 líneas de plan), siete fases de implementación y una auditoría posterior. Además, SDD termina en implementar y no verifica que lo construido cumpla la spec: eso lo cerraron las pruebas y el arranque desde un clon limpio, no la especificación.

Por qué SDD y no BMAD. Se eligió Spec Kit por estar más extendido y ser más útil para el trabajo futuro del equipo. No se probó BMAD, por lo que no hay evidencia propia para compararlos; solo cabe suponer que un reparto por roles daría más estructura de proceso y también más ceremonia.

6.8 Conclusiones

El equipo ahora sabe llevar un requisito con su ID desde el análisis hasta una prueba, detectar las decisiones que el agente tomó sin avisar y llevarlas al cliente, y distinguir una documentación que parece completa de una que se puede comprobar. Queda pendiente: una prueba de integración con PostgreSQL real y Playwright con datos limpios en un entorno repetible, PF-06b, los RNF de operación, corregir el *seed* y el enmascaramiento del RUT, y repetir la usabilidad con liquidadores reales.

Anexo A. Trazabilidad requisito → especificación → tarea → commit → prueba

Universo: 15 requisitos Must de la E1 presentes en las matrices de F1 y F4 (9 RQF y 6 RQNF). RQF-015 no se cuenta: era Should en la E1. Las tareas se agruparon por fase en los commits de avance, de modo que el enlace al commit es por fase y no uno a uno. Fuente: `insumos_informe.md`.

Req.	Spec	Tareas	Commit (fase)	Prueba y estado
RQF-002	F1	F1 T009, T012	`e84179d` (3)	`case-detail.spec.ts`; PF-02 parcial
RQF-004	F4	F4 T005, T010–T014	`6275887`, `e84179d` (1–3)	`scoring-engine.spec.ts`, `scoring-quickstart.spec.ts`; seed con 74,0
RQF-005	F4	F4 T005, T019–T022	`6275887`, `24799d5` (1–2, 5)	`threshold-decision.spec.ts`, `quickstart-validation.spec.ts`
RQF-006	F4	F4 T023–T026	`38f03b9` (6)	`threshold-audit.spec.ts`; **parcial**: PF-06b sin prueba
RQF-007	F1, F4	F1 T018–T019; F4 T015–T016, T021	`181b860` (4)	`fraud-precedence.spec.ts`, `resolution.spec.ts`; no usa los 45 puntos exactos
RQF-008	F1, F4	F1 T018; F4 T015–T018, T021	`181b860` (4)	`fraud-precedence.spec.ts`, `quickstart-validation.spec.ts`; parcial
RQF-009	F1	F1 T009–T015	`e84179d` (3)	`case-detail.spec.ts`, `review-flow.e2e.spec.ts`
RQF-010	F1	F1 T008, T016–T020	`6275887`, `181b860`	`case-resolution.spec.ts`, `review-flow.e2e.spec.ts`; persistencia no demostrada
RQF-011	F1	F1 T016–T020	`181b860` (4)	`case-resolution.spec.ts`, `review-flow.e2e.spec.ts`; resolución simulada
RQNF-001	F1, F4	F1 T001, T004–T005; F4 T001–T002, T007–T008, T027	`6275887`, `450e2ff`	`extra-functional*.e2e-spec.ts`, `sla-concurrency.spec.ts`; sin p95 bajo carga real
RQNF-002	F4	F4 T027	`450e2ff` (7)	`extra-functional.e2e-spec.ts`; **cadena rota**: sin procesamiento asíncrono
RQNF-003	F1, F4	F1 T008b, T027b; F4 T027	`6275887`, `450e2ff`	`pii-masking.spec.ts` y e2e; sin residencia ni egreso reales
RQNF-004	F1	F1 T027b	`450e2ff` (7)	`extra-functional-f1.e2e-spec.ts`; **cadena rota**: hash, no cifrado ni retención
RQNF-005	F1, F4	F1 T008, T016, T027; F4 T005, T023–T026	`6275887`, `181b860`, `450e2ff`	`audit-immutability.e2e-spec.ts`, `threshold-audit.spec.ts`; con mocks
RQNF-006	F1, F4	F1 T027b; F4 T027	`450e2ff` (7)	`extra-functional*.e2e-spec.ts`; **cadena rota**: sin 99,9 % mensual

Anexo B. Bitácora de decisiones

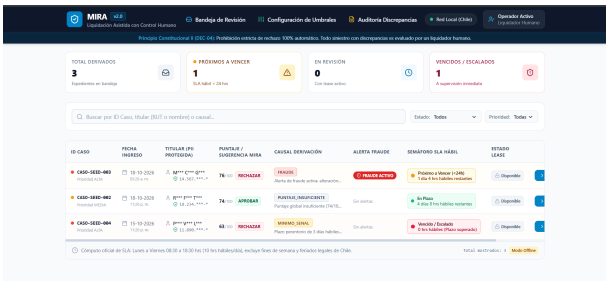
Entrega 1 (numeración del Anexo de la E1): DEC-01 cobro solo de resolución automática concluyente; DEC-02 tramos de más de 15.000 casos; DEC-03 retención por capas (90 días activa, 5 años fría); DEC-04 sin rechazos automáticos; DEC-05 seudonimización con traza inalterable; DEC-06 enmascaramiento de datos personales; DEC-07 ingesta asíncrona (acuse en menos de 1 s); DEC-08 adaptador de carpeta compartida; DEC-09 autenticación federada; DEC-10 MVP acotado a enero de 2027; DEC-11 umbral de aprobación configurable (90 puntos); DEC-12 endpoint de consulta de estado.

Estado en la Entrega 2 (`clarificacion_cambios.md`): intactas DEC-01 a 03, 05 a 10 y 12; ampliadas DEC-04 (el vencimiento no cierra el caso; el fraude prima) y DEC-11 (piso de 60 puntos). Nuevas: **DEC-13** plazo en horas hábiles con escalamiento (17-oct-2026, Q1 y Q3); **DEC-14** concurrencia híbrida con bloqueo de 15 minutos (17-oct-2026, Q5); **DEC-15** MIRA puede sugerir aprobar con puntaje bajo y sin fraude, la decisión sigue siendo humana (18-oct-2026); **DEC-16** el mismo integrante repite como representante del cliente, contra la regla de rotación (18-oct-2026). Fe de erratas: las fichas del cuerpo de la E1 usan numeración desplazada; se cita la del Anexo (`decisiones_entrega2.md`).

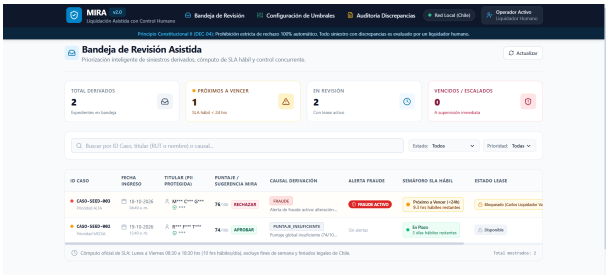
Anexo C. Evidencia del trabajo con el agente

Documentos de evidencia en `docs/entrega2/evidencia/`: `verificacion_clon_limpio.md`, `verificacion_F4.md`, `matriz_pruebas.md`, `verificacion_citas.md`, `autoria.md` e `insumos_informe.md`. Los prompts de la E1 están en `PROMPTS.md`. Para la E2 el flujo de Spec Kit está documentado en `README.md` y las preguntas y respuestas de clarificación en `clarificacion_cambios.md`; no existe una transcripción íntegra de cada prompt escrito para ejecutar los pasos de Spec Kit.

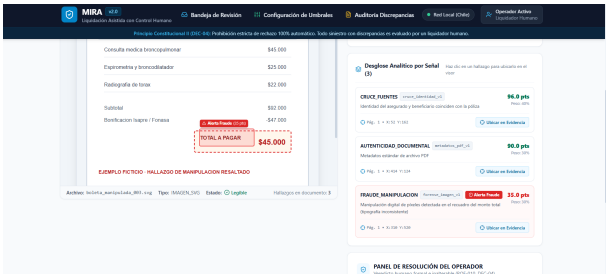
Anexo D. Capturas y evidencia



0. Bandeja antes de cargar el *seed*



1. Bandeja con `CASO-SEED-002` y `003`



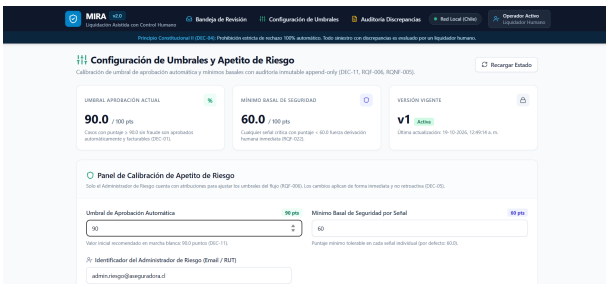
2. Detalle de `CASO-SEED-003` (fraude)



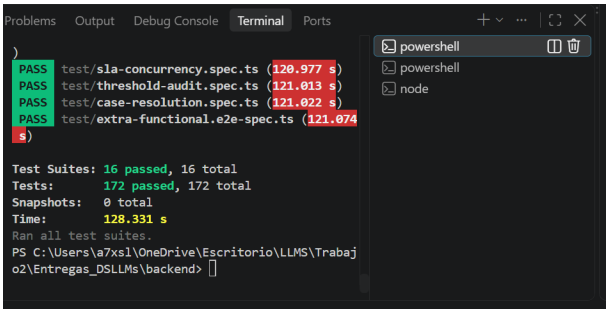
3. Panel de resolución con justificación



4. Discrepancias



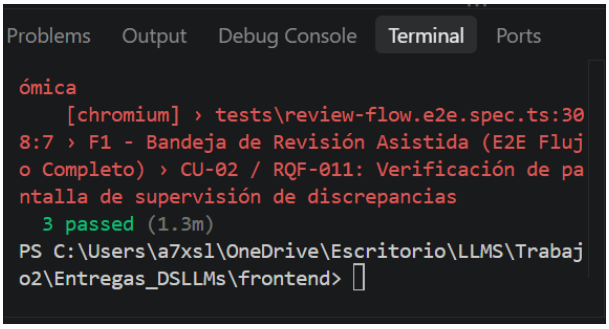
5. Configuración e historial de umbral



6. Pruebas del backend (16 suites, 172 pruebas)

```
>> docker compose ps
NAME            IMAGE                          COMMAND                  SERVICE    CREATED       STATUS                 PORTS
mira-backend    entregas_dsllms-backend        "docker-entrypoint.s…"   backend    3 hours ago   Up 3 hours             0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
mira-db         postgres:16-alpine             "docker-entrypoint.s…"   db         3 hours ago   Up 3 hours (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
mira-frontend   entregas_dsllms-frontend       "docker-entrypoint.s…"   frontend   3 hours ago   Up 3 hours             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\Users\a7xsl\OneDrive\Escritorio\LLMS\Trabajo2\Entregas_DSLLMs\frontend>
```

7. Contenedores en ejecución



8. Pruebas de interfaz (Playwright)

Autoría por carpeta (líneas añadidas/eliminadas; `autoria.md`): `backend` 9.090/104; `frontend` 6.241/72; `specs` 4.198/126; `docs` 3.192/91; `otros` 18.813/1.102; total 41.534/1.495 (43.029). Detalle por commit en `autoria.md`; matriz completa de pruebas en `matriz_pruebas.md`.

Anexo E. Guía simple: qué es SDD y qué técnicas enseña el curso

E.1 ¿Qué es SDD? Imagina que quieres construir una casa. Puedes decirle al maestro "hazme una casa con tres piezas" y corregir sobre la marcha, o puedes pedir primero un plano, revisarlo entre todos y recién entonces construir. Pedir cosas al agente por chat y corregir después es lo primero (en la clase se llama *vibe coding*). **SDD (Spec-Driven Development, desarrollo guiado por especificación)** es lo segundo: antes de generar código se escribe una *especificación* ("spec") que dice *qué* debe hacer el sistema, y el código se deriva de ella.

	Pedir por chat (vibe coding)	**SDD**
Dónde vive el contexto	En el chat, y se pierde	En archivos versionados en el repositorio
Qué pasa con lo ambiguo	El agente rellena con supuestos	Se marca (`[NEEDS CLARIFICATION]`) y se pregunta
Alcance	Se renegocia en cada vuelta	Se congela antes de generar
Qué se revisa antes del código	Nada	La spec y el plan, como si fueran código
Costo (según el curso)	Más correcciones manuales	Cerca de 20–40 % más de tokens por funcionalidad, a cambio de menos ciclos de corrección

Fuente: material del curso, módulo "Spec-Driven Development" (Prof. Diego Gatica).

E.2 GitHub Spec Kit. Es un conjunto de herramientas de código abierto (licencia MIT) que ordena ese flujo. *No es un agente ni un modelo*: corre dentro del agente que cada persona usa (Copilot Chat, Claude Code, Codex, Gemini CLI, Cursor). Se instala con un programa de línea de comandos (`specify`) que deja en el repositorio plantillas y comandos: `.specify/` (plantillas y la *constitución* en `memory/constitution.md`) y una carpeta `specs/<funcionalidad>/` con `spec.md`, `plan.md` y `tasks.md`. Termina en "implementar" y *no* comprueba que lo construido cumpla la spec: eso lo cierran las pruebas.

E.3 Las etapas, en palabras simples.

Comando	Pregunta que responde	Qué produce
`/speckit.constitution`	¿Qué reglas no se pueden romper nunca?	`constitution.md`: principios del proyecto, que todas las etapas leen antes de proponer algo
`/speckit.specify`	¿*Qué* queremos? (sin elegir tecnología)	`spec.md`: comportamiento esperado y criterios de aceptación
`/speckit.clarify`	¿Qué falta o es ambiguo?	Preguntas de a una, con opciones; las respuestas vuelven a la spec
`/speckit.plan`	¿*Cómo* lo construimos?	`plan.md`, `research.md`, `data-model.md`: arquitectura, contratos y datos (aquí se elige el stack)
`/speckit.tasks`	¿Qué hay que hacer, en qué orden?	`tasks.md`: tareas ordenadas por dependencia
`/speckit.analyze`	¿Spec, plan y tareas se contradicen?	Reporte de consistencia antes de programar
`/speckit.implement`	Construir	Código y pruebas, marcando las tareas hechas
`/speckit.checklist`, `/speckit.converge`	(opcionales) ¿Está completa la spec? / ¿qué falta respecto del código existente?	Lista de verificación / tareas añadidas al final de `tasks.md`

La sintaxis cambia según el agente: en Codex se escribe `$speckit-plan` y en Copilot Chat o Claude Code `/speckit.plan`. El flujo es el mismo.

E.3b Plan explícito frente a "razonar y actuar". El curso distingue dos formas de controlar a un agente: *ReAct* (razona y actúa paso a paso; flexible pero difícil de auditar porque el plan nunca se escribe) y *plan-and-execute* (primero un plan revisable, luego la ejecución; más trazable). SDD es plan-and-execute y es la base de la trazabilidad spec → plan → código → pruebas.

E.4 Técnicas de prompting del curso.

Técnica	**En palabras simples**
1. Rol y contexto	Decirle al modelo quién es, en qué sistema trabaja y con qué tecnología.
2. Especificidad y delimitadores	Pedir algo preciso (qué, con qué reglas, cómo se acepta) y separar los datos de las instrucciones con marcas claras.
3. Ejemplos (few-shot)	Mostrar 2 a 5 ejemplos de la salida buscada y un contraejemplo de lo que *no* sirve.
4. Razonamiento explícito	Pedir los pasos intermedios (restricciones, alternativas, comparación) antes de la conclusión.
5. Salida estructurada	Exigir un formato fijo (JSON, tabla) para poder validarla y compararla.
6. Descomposición y encadenamiento	Dividir una tarea grande en pasos chicos, cada uno con su entrada, salida y criterio.
7. Autocrítica y evaluación por modelo	Que el modelo (u otro) revise la salida contra criterios escritos, y muestrear a mano.
8. Prompting para código	Dar el test que debe pasar, el contexto del repositorio, pedir el plan antes del código y fijar criterios de aceptación.
Grounded prompting (anclaje)	Separar cada fuente con un identificador, citar la fuente de cada afirmación y declarar lo que no tiene fuente en vez de rellenarlo.
Structured output	Esquema fijo: lo que falta aparece como campo vacío en vez de desaparecer.
Plan-and-Solve	Fase A: solo planificar; se revisa el plan; fase B: ejecutar un paso a la vez.
Chain-of-Verification	El modelo formula preguntas de verificación sobre su propia salida, las responde *aparte*, sin ver el texto original, y corrige lo que no resiste.
LLM-as-a-Judge	Un segundo prompt evalúa con una rúbrica de criterios sí/no, cada falla con su evidencia y sin nota global.

Anexo F. Dónde se usó cada técnica en MIRA

Solo se listan usos que tienen evidencia en el repositorio (commits y archivos) o en los prompts guardados. La columna de comando distingue lo que los informes *declaran* de lo que los commits *prueban*: un commit muestra qué archivo cambió, no qué comando lo generó. Lo que no tiene evidencia figura como pendiente o como "no usada"; no se declara una técnica que el equipo no aplicó.

Técnica / etapa	**Para qué se usó y evidencia**	**Comando o prompt**
Constitución	Principios no negociables con los DEC de la E1 que aplican (por ejemplo DEC-04, "nunca rechazo automático", visible como Principio II en la interfaz). `.specify/memory/constitution.md` v2.0.0, actualizada en el commit `03d39bf` (16-oct-2026 17:42, Frxnks)	*Sin evidencia de que se ejecutara el comando*; el commit solo muestra que se añadieron decisiones al archivo
Especificar	Una spec por funcionalidad, conservando los RQF de la E1: `specs/001-bandeja-revision-asistida`, `specs/002-puntaje-decision-umbrales`. Commit `184d9a3` (17-oct 02:05, Frxnks)	`/speckit-specify`, declarado en el informe de análisis; el commit no registra el comando
Aclarar	Preguntas Q1–Q5 respondidas con el representante y vueltas a las specs: commit `8310291` (17-oct 04:53); documentación en `docs/entrega2/clarificacion_cambios.md`, commit `dc16bb1` (17-oct 13:07). Autor Frxnks	`/speckit-clarify`, declarado en `clarificacion_cambios.md`
Plan	`plan.md`, `data-model.md` y `quickstart.md` de ambas funcionalidades: commit `a334b86` (17-oct 17:14, Frxnks)	`/speckit-plan`, declarado en el informe de análisis
Tareas	`tasks.md` de ambas funcionalidades: commit `253fd9a` (17-oct 18:00, Frxnks); 58 tareas iniciales, 61 tras el análisis	`/speckit-tasks`, declarado en el informe de análisis
Analizar	Reporte de consistencia con 11 hallazgos y ajustes a `specs/`: `docs/entrega2/analisis_cruzado_especificaciones.md`, commit `0b151a5` (17-oct 20:20, Frxnks)	`/speckit-analyze`, declarado explícitamente en el informe
Implementar	Fases 1 y 2 en `6275887` (T001–T009, 17-oct 21:53) y fases 3 a 7 en `e84179d`, `181b860`, `24799d5`, `38f03b9` y `450e2ff` ("Avance a fase N", 17-oct 22:08 a 23:26, Frxnks). Cada commit de avance marca tareas de `tasks.md` como completadas	*Sin evidencia de ejecución de* `/speckit-implement`; los commits no identifican el comando.
Checklist y converge	No usadas: no hay registro de ejecución ni artefacto atribuible. Los `checklists/requirements.md` que aparecen en el commit de las specs son los integrados por la herramienta, no los de `/speckit-checklist`	–
Rol y contexto (técnica 1)	Los prompts de auditoría abren con el contexto del proyecto (MIRA, método SDD, funcionalidades F1 y F4, stack, fecha de evaluación) y reglas numeradas de conducta del agente. Redactados con apoyo de Claude y ejecutados en	Prompts de la E1 en `PROMPTS.md`; los de la E2 no están transcritos completos (Anexo C)

Anexo G. Cómo obtener y ejecutar el repositorio

1. Clonar. Repositorio: `https://github.com/Frxnks/Entregas_DSLLMs`

```
git clone https://github.com/Frxnks/Entregas_DSLLMs.git
cd Entregas_DSLLMs
```

2. Dos documentos en la raíz del repositorio explican cómo usarlo:

- `README.md`: explicación más completa (componentes, requisitos, arranque, datos de demostración, API, pruebas automatizadas, detener y reiniciar).
- `prueba de funcionamiento.md`: guía más directa de cómo usar la aplicación.

3. Resumen del arranque (requiere Docker Desktop con Compose v2 y los puertos 3000, 3001 y 5432 libres; el detalle está en el `README.md`):

```
docker compose up -d --build
docker compose exec backend node dist/prisma/seed.js
```

Después se abre `http://localhost:3000` (redirige a `/bandeja`). El *seed* carga los casos ficticios `CASO-SEED-001`, `002` y `003` y borra los datos previos de esa base; no se debe usar `docker compose down -v` salvo que se quiera borrar la base. Versión evaluada: etiqueta `v2.0`.

Anexo H. Sesión de aceptación: criterios de la E1 y evidencia disponible

La columna de evidencia resume `matriz_pruebas.md`, `verificacion_F4.md` y la comprobación manual del 18-oct-2026. La columna de resultado reproduce lo marcado en `sesion_aceptacion.txt`; "Sin marcar" significa que la sesión no registró resultado y no equivale a aceptado. La hoja no registra fecha ni asistentes.

Criterio	Qué se comprueba (E1)	Evidencia disponible	Resultado
PF-02	Evidencia ilegible: causa clara y el resto sigue visible	Parcial: una evidencia ilegible, no el expediente de tres archivos	Sin marcar
PF-04	Puntaje de 0 a 100 con desglose	Cubierto en lógica; `CASO-SEED-002`: 74,25 calculado frente a 74 guardado	Sin marcar
PF-05	Umbral 90, puntaje alto sin fraude: aprueba solo	Cubierto en lógica; por API, 94 → aprobado automático	Sin marcar
PF-06	Cambio de umbral con usuario, fecha, valor anterior y versión	Parcial con mocks; manual 90→88→90 con fila de auditoría	Sin marcar
PF-06b	Rechazar publicar un flujo sin módulos activos	Sin prueba ni criterio en la spec F4	Rechazado (falta de evidencia)
PF-07	Puntaje bajo el umbral: no hay rechazo automático	Parcial; por API, 80 → derivado (no usa los 45 puntos exactos)	Sin marcar
PF-08	Fraude con puntaje alto: deriva igual	Parcial; por API, fraude con 99 → derivado	Sin marcar
PF-09	Evidencia, puntaje, señales y razón en una pantalla	Parcial; comprobado a mano en el detalle del caso	Sin marcar
PF-10	Decisión inalterable con fecha, hora y autor	Parcial con mocks; manual: `CASO-SEED-002` quedó `RESUELTO`	Sin marcar
PF-11	Contradecir a MIRA conserva la decisión y marca discrepancia	Parcial: la interfaz simula la respuesta en Playwright	Sin marcar
PF-15	Solo la aprobación automática es facturable	Parcial; por API, verdadero en A y falso en B, C y D	Sin marcar
PX-01	Rendimiento de la API bajo carga	Mediciones con servicios simulados, sin p95 bajo carga real	Sin marcar
PX-02	Expediente pesado: acuse en menos de 1 s	No demostrado (sin procesamiento asíncrono)	Aceptado con observación
PX-03	Ningún RUT ni nombre completo sale a modelos externos	Solo pruebas unitarias de enmascaramiento; sin inspección de tráfico	Sin marcar
PX-04	Recuperación desde almacenamiento frío en 10 s	No demostrado (solo hash y metadatos)	Aceptado con observación
PX-05	Auditoría inalterable incluso para el administrador	Parcial: middleware con mocks	Sin marcar
PX-06	Caída de proveedor de 3 horas sin pérdida de casos	No demostrado	Aceptado con observación
PX-07	Carga sostenida de 15.000 casos al mes	Sin prueba de carga	Sin marcar
PX-08	Pantalla en menos de 2 s; operador sin capacitación	`/bandeja` cargó en 477 ms; usabilidad en 6.5	Rechazado (falta de evidencia)